

LAPORAN SOFTWARE QUALITY ASSURANCE

METODE WHITEBOX – FORMAL INSPECTION



Anggota kelompok:

20231310046
20231310047

M. Irvan Alfiansyah
M. Nur Yanfa

PROGRAM STUDI TEKNIK INFORMATIKA
UNIVERSITAS KEBANGSAAN REPUBLIK INDONESIA

*Jln. Terusan Halimun No. 37 (Pelajar Pejuang 45) Lingkar Selatan Kec. Lengkong
kota Bandung Jawa Barat 40263*

1. DASAR TEORI FORMAL INSPECTION

Formal Inspection adalah metode white box testing berbasis pemeriksaan sistematis terhadap source code menggunakan checklist yang terstruktur. Setiap aspek penting diperiksa secara konsisten oleh reviewer yang bertindak sebagai inspektur independen.

Berbeda dari unit test yang berfokus pada input-output, Formal Inspection mengevaluasi kualitas internal program dari berbagai dimensi:

- 1. Struktur layer dan pemisahan tanggung jawab (separation of concerns)
- 2. Validasi akses dan mekanisme autentikasi
- 3. Validasi dan sanitasi data masukan
- 4. Integritas transaksi database (ACID properties)
- 5. Penanganan error yang terklasifikasi dengan benar
- 6. Keamanan query database (SQL injection prevention)
- 7. Konsistensi status objek domain
- 8. Keterlacakan dan jejak audit (auditability)
- 9. Identifikasi risiko implementasi

Metode ini dipilih untuk modul Checkout ACID karena checkout adalah proses kritis yang secara langsung mempengaruhi integritas stok, data pembayaran, dan rekaman order. Kesalahan pada satu titik dapat berdampak finansial dan operasional.

2. RUANG LINGKUP INSPEKSI

Inspeksi mencakup seluruh kode yang langsung mendukung fitur Checkout ACID. Pengujian tidak mencakup audit keamanan menyeluruh seluruh aplikasi; beberapa catatan hardening produksi dicantumkan secara terpisah sebagai risiko di luar scope inti.

ID Black-Box	Skenario	Dicakup	Bukti Pemeriksaan
CHK-01	Checkout berhasil	Ya	Handler, usecase, repository transaction
CHK-02	Stok tidak cukup	Ya	Validasi stok produk/varian
CHK-03	Race condition checkout bersamaan	Ya	Locking FOR UPDATE di repository
CHK-04	Cart kosong	Ya	Guard cart kosong di usecase
CHK-05	Midtrans API error	Ya	snapTokenFn, error log, order tetap return
CHK-06	Checkout tanpa JWT/session	Ya	Handler membaca user_id dari context JWT

ID Black-Box	Skenario	Dicakup	Bukti Pemeriksaan
CHK-07	Atomicity stok/order	Ya	db.Transaction dan rollback otomatis
CHK-08	Status order PENDING	Ya	Repository membuat order dengan status PENDING

3. SOURCE CODE YANG DIINSPEKSI

No	File	Baris	Bagian Kode	Item Inspeksi
1	<i>order_handler.go</i>	15-26	classifyCheckoutError	Mapping error bisnis dan gateway
2	<i>order_handler.go</i>	60-87	OrderHandler.Checkout	Auth, session, response sukses/error
3	<i>order_usecase.go</i>	72-99	orderUsecase.Checkout	Cart, transaction, Midtrans handling
4	<i>order_repository.go</i>	23-120	CheckoutTransaction	Transaction, stok, order, cart cleanup
5	<i>order_repository.go</i>	37, 45	Locking FOR UPDATE	Isolation checkout bersamaan
6	<i>order_repository.go</i>	48-64	Validasi & pengurangan stok	Consistency stok, cegah stok negatif
7	<i>order_repository.go</i>	71-80	Order item snapshot	Konsistensi harga dan item
8	<i>order_repository.go</i>	86-95	Pembuatan order	Status awal dan relasi user
9	<i>order_repository.go</i>	101-108	Create items dan delete cart	Atomic completion
10	<i>order_usecase.go</i>	286-333	BatchProcessSupplierOrders	Loop/filter order pasca-checkout + audit log

4. KRITERIA CHECKLIST INSPEKSI

Checklist disusun berdasarkan kebutuhan transaksi checkout, prinsip ACID, dan standar keamanan dasar. Setiap kode kriteria mewakili satu dimensi kualitas yang diperiksa secara independen.

Kode Kriteria	Area	Pertanyaan Utama
AUTH	Authentication / Authorization	Apakah checkout hanya dapat dilakukan oleh user yang valid dan terautentikasi?
INPUT	Input & Request	Apakah input tidak dipercaya secara berlebihan? Apakah user dapat memanipulasi identitas dari body?
CART	Cart Validation	Apakah cart kosong atau cart error ditangani dengan benar sebelum transaksi dimulai?
TX	Database Transaction	Apakah semua operasi penting (stok, order, cart) berada dalam satu transaksi atomik?
STOCK	Stock Consistency	Apakah stok selalu dicek dan divalidasi sebelum dikurangi? Apakah stok dapat menjadi negatif?
ISO	Isolation	Apakah update stok dilindungi dari race condition saat checkout bersamaan?
ORDER	Order Consistency	Apakah order dan order item dibuat secara konsisten dengan snapshot harga yang benar?
ERROR	Error Handling	Apakah error bisnis dan error server dipisahkan dengan status HTTP yang tepat?
PAY	Payment Handling	Apakah kegagalan Midtrans tidak merusak atau menghilangkan order internal yang sudah dibuat?
SEC	Security Baseline	Apakah semua query database memakai parameter binding untuk mencegah SQL injection?
AUDIT	Auditability	Apakah perubahan status order pasca-checkout dapat dicatat untuk keperluan audit?

5. CHECKLIST INSPEKSI DETAIL

Seluruh 25 butir checklist diperiksa secara independen terhadap source code aktual. Setiap baris mencatat lokasi kode, ekspektasi, temuan aktual, dan status akhir.

5.1 Kelompok AUTH — Autentikasi & Otorisasi

ID	Kriteria	Source Code	Ekspektasi	Hasil Pemeriksaan	Status
FI-01	AUTH	<i>order_handler.go:62-66</i>	user_id wajib ada di context	Handler return 401 Unauthorized jika user_id tidak ada	PASS
FI-02	AUTH	<i>order_handler.go:67-71</i>	user_id harus bertipe string	Handler return 401 jika type assertion gagal	PASS
FI-03	INPUT	<i>order_handler.go:73-79</i>	User tidak dapat mengirim user_id dari body	Body hanya membaca voucher_code; user_id dari JWT context	PASS

5.2 Kelompok CART — Validasi Keranjang

ID	Kriteria	Source Code	Ekspektasi	Hasil Pemeriksaan	Status
FI-04	CART	<i>order_usecase.go:73-76</i>	Error cart repository menghentikan checkout	Return 'gagal memuat keranjang belanja' jika repo error	PASS
FI-05	CART	<i>order_usecase.go:78-80</i>	Cart kosong ditolak sebelum transaksi	Return error 'keranjang belanja kosong'	PASS

5.3 Kelompok TX — Transaksi Database

ID	Kriteria	Source Code	Ekspektasi	Hasil Pemeriksaan	Status
FI-06	TX	<i>order_repository.go:27</i>	Seluruh operasi checkout atomik	Menggunakan r.db.Transaction — semua atau tidak sama sekali	PASS
FI-15	TX	<i>order_repository.go:97-104</i>	Order dan item dibuat dalam transaction	tx.Create masih berada di dalam callback transaction	PASS
FI-16	TX	<i>order_repository.go:107</i>	Cart dihapus setelah order sukses	Delete CartItem masih dalam transaction yang sama	PASS
FI-17	TX	<i>order_repository.go:115-117</i>	Error menyebabkan rollback otomatis	GORM rollback saat error dikembalikan dari callback	PASS

5.4 Kelompok ISO & STOCK — Isolation dan Konsistensi Stok

ID	Kriteria	Source Code	Ekspektasi	Hasil Pemeriksaan	Status
FI-07	ISO	<i>order_repository.go:37</i>	Produk dikunci saat checkout	<code>clause.Locking{Strength:"UPDATE"}</code> pada produk	PASS
FI-08	ISO	<i>order_repository.go:45</i>	Varian dikunci saat checkout	<code>clause.Locking{Strength:"UPDATE"}</code> pada varian	PASS
FI-09	STOCK	<i>order_repository.go:48-54</i>	Stok varian dicek sebelum dikurangi	Return error jika <code>variant.Stock < item.Quantity</code>	PASS
FI-10	STOCK	<i>order_repository.go:58-65</i>	Stok produk dicek sebelum dikurangi	Return error jika <code>product.Stock < item.Quantity</code>	PASS
FI-11	STOCK	<i>order_repository.go:51-63</i>	Stok tidak boleh negatif	Pengurangan dilakukan setelah validasi — tidak pernah di bawah nol	PASS

5.5 Kelompok ORDER — Konsistensi Order

ID	Kriteria	Source Code	Ekspektasi	Hasil Pemeriksaan	Status
FI-12	ORDER	<i>order_repository.go:69</i>	Total dihitung dari harga saat checkout	<code>totalAmount += priceAtPurchase * quantity</code>	PASS
FI-13	ORDER	<i>order_repository.go:71-80</i>	Order item menyimpan snapshot harga	<code>PriceAtPurchase</code> disimpan per item	PASS
FI-14	ORDER	<i>order_repository.go:86-95</i>	Order punya user dan status yang benar	<code>UserID=userID, Status="PENDING"</code> saat create	PASS

5.6 Kelompok ERROR & PAY — Penanganan Error dan Pembayaran

ID	Kriteria	Source Code	Ekspektasi	Hasil Pemeriksaan	Status
FI-18	ERROR	<i>order_handler.go:15-26</i>	Cart/stok bukan server error	Dipetakan ke 400 Bad Request — bukan 500	PASS
FI-19	ERROR	<i>order_handler.go:22-23</i>	Payment gateway error menjadi 502	Dipetakan ke 502 Bad Gateway — bukan 500	PASS
FI-20	PAY	<i>order_usecase.go:89-96</i>	Midtrans error tidak crash atau batalkan order	Error hanya di-log; order tetap di-return ke client	PASS

ID	Kriteria	Source Code	Ekspektasi	Hasil Pemeriksaan	Status
FI-21	PAY	<i>order_usecase.go:90-93</i>	Token sukses disimpan ke database	SavePaymentToken dipanggil jika snapErr == nil	PASS

5.7 Kelompok SEC & AUDIT — Keamanan dan Keterlacakan

ID	Kriteria	Source Code	Ekspektasi	Hasil Pemeriksaan	Status
FI-22	SEC	<i>order_repository.go:37</i>	Query product memakai parameter binding	Where("id_product = ?", item.ProductID)	PASS
FI-23	SEC	<i>order_repository.go:45</i>	Query variant memakai parameter binding	Where("id_variant = ?", *item.VariantID)	PASS
FI-24	SEC	<i>order_repository.go:107</i>	Delete cart dibatasi per user	Where("id_user = ?", userID) — tidak bisa hapus cart user lain	PASS
FI-25	AUDIT	<i>order_usecase.go:313-323</i>	Batch supplier mencatat audit log	Audit log dibuat untuk setiap order valid dalam batch supplier	PASS

Ringkasan: Seluruh 25 butir checklist (FI-01 s.d. FI-25) mendapat status PASS. Tidak ada temuan defect kritis pada core transaksi Checkout ACID.

6. BUKTI SOURCE CODE KRITIS

6.1 Bukti Transaksi Atomik (TX)

Seluruh operasi inti — pembuatan order, order item, pengurangan stok, dan penghapusan cart — berada dalam satu callback `db.Transaction`. Jika salah satu operasi mengembalikan error, GORM secara otomatis melakukan rollback.

```
err := r.db.Transaction(func(tx *gorm.DB) error {
    // ... validasi dan pengurangan stok ...

    if err := tx.Create(&createdOrder).Error; err != nil {
        return err // trigger ROLLBACK
    }
    if err := tx.Create(&orderItems).Error; err != nil {
        return err // trigger ROLLBACK
    }
    if err := tx.Where("id_user = ?", userID).Delete(&domain.CartItem{}).Error;
err != nil {
        return err // trigger ROLLBACK
    }
    return nil // trigger COMMIT
})
```

6.2 Bukti Locking Stok (ISO)

Baris produk dan varian dikunci menggunakan `FOR UPDATE` sebelum stok dibaca dan dikurangi. Hal ini memastikan dua checkout bersamaan tidak dapat membaca stok yang sama secara tidak terkendali.

```
// Lock produk
tx.Clauses(clause.Locking{Strength: "UPDATE"}).
    Where("id_product = ?", item.ProductID).
    First(&product)

// Lock varian (jika ada)
tx.Clauses(clause.Locking{Strength: "UPDATE"}).
    Where("id_variant = ?", *item.VariantID).
    First(&variant)
```

6.3 Bukti Validasi Stok (STOCK)

Pengurangan stok hanya dilakukan setelah validasi berhasil. Jika stok tidak mencukupi, fungsi mengembalikan error dan transaction melakukan rollback — stok tidak pernah menjadi negatif.

```
if product.Stock < item.Quantity {
    return fmt.Errorf("stok produk '%s' tidak mencukupi. Stok tersisa: %d",
        product.Name, product.Stock)
}
product.Stock -= item.Quantity // pengurangan hanya setelah validasi
tx.Save(&product)
```

6.4 Bukti Payment Handling (PAY)

Pembuatan Snap Token Midtrans adalah proses eksternal yang dapat gagal secara independen. Kegagalan token hanya dicatat (log) dan tidak membatalkan order internal yang sudah tersimpan di database.

```
snapResp, snapErr := u.snapTokenFn(order.ID, order.TotalAmount)
if snapErr == nil && snapResp != nil {
    order.PaymentToken = &snapResp.Token
    order.PaymentURL    = &snapResp.RedirectURL
    u.orderRepo.SavePaymentToken(order.ID, snapResp.Token,
        snapResp.RedirectURL)
} else if snapErr != nil {
    // Log only – order internal TIDAK dibatalkan
    fmt.Printf("[MIDTRANS ERROR] Gagal generate Snap Token: %v\n", snapErr)
}
return order, nil // order selalu dikembalikan
```

6.5 Bukti Security Baseline (SEC)

Semua query kritis menggunakan parameter binding GORM yang mencegah SQL injection. Scope Delete cart dibatasi ke userID milik session aktif.

```
// Parameter binding – bukan string concatenation
Where("id_product = ?", item.ProductID) // FI-22
Where("id_variant = ?", *item.VariantID) // FI-23
Where("id_user = ?", userID) // FI-24: scope delete per-user
```

7. TEST PENDUKUNG INSPEKSI

7.1 Command Eksekusi

```
cd backend-go
go test ./internal/usecase -run "TestCheckout|TestDataFlow|TestLoopTesting" -v
```

7.2 Hasil Eksekusi

```
=== RUN    TestCheckout_CartRepoError
--- PASS: TestCheckout_CartRepoError
=== RUN    TestCheckout_EmptyCart
--- PASS: TestCheckout_EmptyCart
=== RUN    TestCheckout_TransactionFails_StokKurang
--- PASS: TestCheckout_TransactionFails_StokKurang
=== RUN    TestCheckout_Success
--- PASS: TestCheckout_Success
=== RUN    TestCheckout_MidtransFails_OrderTetapKembali
--- PASS: TestCheckout_MidtransFails_OrderTetapKembali
=== RUN    TestCheckout_SnapRespNil_OrderTetapKembali
--- PASS: TestCheckout_SnapRespNil_OrderTetapKembali
=== RUN    TestDataFlow_CheckoutCartToOrderAndPaymentToken
--- PASS: TestDataFlow_CheckoutCartToOrderAndPaymentToken
=== RUN    TestLoopTesting_CheckoutCartItemCounts
--- PASS: TestLoopTesting_CheckoutCartItemCounts
=== RUN    TestLoopTesting_BatchProcessSupplierOrders
--- PASS: TestLoopTesting_BatchProcessSupplierOrders
=== RUN    TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder
--- PASS: TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder
PASS
ok      github.com/nuryanfa/e-commerse-sqa/internal/usecase
```

7.3 Pemetaan Test ke Checklist

Nama Test	Checklist yang Didukung	Skenario Tervalidasi	Status
<i>TestCheckout_CartRepoError</i>	FI-04	Error cart repo tidak lanjut	PASS
<i>TestCheckout_EmptyCart</i>	FI-05	Cart kosong ditolak (CHK-04)	PASS
<i>TestCheckout_TransactionFails_StokKurang</i>	FI-09, FI-10, FI-17	Stok kurang, transaction rollback	PASS
<i>TestCheckout_Success</i>	FI-06, FI-12, FI-13, FI-14	Checkout normal, order terbuat	PASS
<i>TestCheckout_MidtransFails_OrderTetapKembali</i>	FI-20	Midtrans gagal, order aman	PASS
<i>TestCheckout_SnapRespNil_OrderTetapKembali</i>	FI-20	Snap nil, tidak crash	PASS
<i>TestDataFlow_CheckoutCartToOrderAndPaymentToken</i>	FI-21	Token & URL tersimpan	PASS
<i>TestLoopTesting_CheckoutCartItemCounts</i>	FI-05, FI-15	Variasi jumlah cart item	PASS
<i>TestLoopTesting_BatchProcessSupplierOrders</i>	FI-25	Audit log batch supplier	PASS

8. ANALISIS ACID HASIL INSPEKSI

Prinsip ACID	Status	Bukti Source Code	Penjelasan
Atomicity	Terpenuhi	<i>r.db.Transaction(...)</i> — semua atau tidak sama sekali	Stok, order, order item, dan cart cleanup dalam satu unit transaksi
Consistency	Terpenuhi	<i>Validasi stok sebelum dikurangi; PriceAtPurchase; Status PENDING</i>	Data order tidak dibuat jika stok tidak cukup; harga di-snapshot
Isolation	Terpenuhi	<i>clause.Locking{Strength: "UPDATE"} pada produk & varian</i>	Dua checkout bersamaan tidak bebas mengurangi stok yang sama
Durability	Terpenuhi	<i>tx.Create(&createdOrder), tx.Create(&orderItems) → PostgreSQL</i>	Setelah COMMIT, order tersimpan permanen dan tidak hilang saat restart

9. ANALISIS RISIKO

Risiko	Kondisi Saat Ini	Dampak	Status	Prioritas
Overselling saat checkout bersamaan	Ada FOR UPDATE pada produk & varian	Risiko berkurang signifikan	Terkendali	Rendah
Order dibuat tanpa item	Cart kosong ditolak; order item dalam transaction	Risiko sangat rendah	Terkendali	Rendah
Stok minus / negatif	Validasi stok sebelum pengurangan	Risiko sangat rendah	Terkendali	Rendah
Transaksi parsial (partial commit)	GORM transaction rollback otomatis saat error	Risiko sangat rendah	Terkendali	Rendah
Kegagalan Midtrans tidak terdeteksi client	Order tersimpan, error di-log, tidak ada retry otomatis	Risiko operasional — client tidak punya link pembayaran	Perlu Monitor	Sedang
Mapping error berbasis string matching	classifyCheckoutError memakai strings.Contains	Rapuh jika pesan error berubah	Perlu Perbaikan	Sedang
Voucher tidak aktif tetapi parameter masih ada	voucherCode diteruskan antar layer tanpa implementasi	Membingungkan dokumentasi dan pembaca kode	Perlu Catatan	Rendah

10. CATATAN HARDENING DI LUAR SCOPE FORMAL INSPECTION

Formal Inspection ini difokuskan pada core transaksi Checkout ACID. Beberapa aspek berikut memerlukan pengujian keamanan terpisah pada fase hardening produksi:

10. CORS pada environment production harus dibatasi hanya ke domain resmi — bukan wildcard.
11. Webhook Midtrans perlu validasi signature untuk mencegah pemalsuan notifikasi pembayaran.
12. Endpoint Swagger/OpenAPI sebaiknya dibatasi aksesnya atau dihapus pada environment production.
13. Rate limiting dapat diperluas ke endpoint checkout dan seluruh endpoint sensitif lainnya.
14. File konfigurasi .env tidak boleh masuk ke version control repository.
15. Upload file perlu validasi MIME type, ukuran maksimal, dan ekstensi yang diizinkan.
16. JWT secret harus berupa nilai acak yang kuat dan unik untuk setiap environment production.

11. TRACEABILITY MATRIX

Tabel berikut menunjukkan pemetaan lengkap antara butir checklist Formal Inspection, skenario black-box yang direferensikan, bukti kode, dan status akhir.

Checklist	Skenario Black-Box	Bukti Source Code	Jumlah Item	Status
FI-01 s.d. FI-03	CHK-06 (Tanpa JWT)	<i>Handler auth context — c.Get("user_id")</i>	3	PASS
FI-04 s.d. FI-05	CHK-04 (Cart kosong)	<i>Usecase cart validation — len(cartItems)</i>	2	PASS
FI-06 s.d. FI-17	CHK-01, 02, 03, 07, 08	<i>Repository transaction, lock, stok, order</i>	12	PASS
FI-18 s.d. FI-19	CHK-02, 04, 05	<i>classifyCheckoutError — HTTP mapping</i>	2	PASS
FI-20 s.d. FI-21	CHK-05 (Midtrans error)	<i>snapTokenFn, log, SavePaymentToken</i>	2	PASS
FI-22 s.d. FI-24	Security baseline	<i>Parameter binding GORM — Where("? = ?", val)</i>	3	PASS
FI-25	Pasca-checkout supplier	<i>BatchProcessSupplierOrders — audit log</i>	1	PASS

12. RINGKASAN EKSEKUTIF

Kategori	Nilai	Status
Total Butir Checklist	25 butir (FI-01 s.d. FI-25)	Semua Tereksekusi
Checklist PASS	25 / 25 (100%)	PASS
Checklist FAIL / Defect Kritis	0	Tidak Ada
Prinsip ACID Terpenuhi	4 / 4 (Atomicity, Consistency, Isolation, Durability)	Terpenuhi Penuh
Unit Test Pendukung	10 / 10 PASS	PASS
Risiko Kritis (core transaksi)	0 risiko kritis	Terkendali
Risiko Perlu Monitoring/Perbaikan	2 (Midtrans retry, string-based error mapping)	Sedang
Catatan Hardening di Luar Scope	7 poin (CORS, webhook, secret, dsb.)	Perlu Security Review

13. KESIMPULAN

Formal Inspection pada fitur Checkout ACID dinyatakan **LULUS** berdasarkan seluruh 25 butir checklist (FI-01 s.d. FI-25) yang mendapat status PASS.

Source code checkout telah memenuhi seluruh aspek berikut sesuai hasil inspeksi:

17. Autentikasi checkout dilakukan melalui JWT context — user tidak dapat memanipulasi identitas dari body request.
18. Cart kosong ditolak di level usecase sebelum transaksi database dimulai.
19. Seluruh operasi inti berada dalam satu transaksi atomik (db.Transaction) dengan rollback otomatis.
20. Validasi stok produk dan varian dilakukan sebelum pengurangan — stok tidak dapat menjadi negatif.
21. Row locking (FOR UPDATE) melindungi dari race condition pada checkout bersamaan.
22. Order dan order item dibuat secara konsisten dengan snapshot harga (PriceAtPurchase).
23. Cart dihapus dalam transaksi yang sama — tidak ada cart tersisa setelah order berhasil.
24. Error bisnis dipetakan ke HTTP 400/409/502 — tidak ada unhandled 500 pada kondisi yang dapat diantisipasi.
25. Kegagalan Midtrans tidak membatalkan order internal — order tetap durable.
26. Seluruh query utama menggunakan parameter binding GORM — baseline SQL injection prevention terpenuhi.

Risiko tersisa yang memerlukan tindak lanjut bukan berada pada core transaksi Checkout ACID, melainkan pada hardening production (CORS, webhook signature, secret management, retry payment token, dan typed error). Seluruh poin tersebut direkomendasikan untuk ditangani dalam Security Review terpisah sebelum go-live.